A Project Report

on

IMPLEMENTATION OF CRYPTO PROCESSOR USING AES-RSA TECHNIQUES FOR SECURING VISUAL DATA INTEGRITY

Submitted in partial fulfillment of the requirement for the award of degree

of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

by

KALAGADDA SRAVANI	-	**22FE1A0456**
KONDARPU RESHMA	-	**22FE1A0472**
PILLA JAYASREE	-	**22FE1A0454**
KANAPARTHI SYAM SUNDUR	-	**22FE1A0457**

Under the esteemed guidance of

Mr. CH. RAGHUNATHA BABU, **M. Tech, (PhD)**

Associate Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VIGNAN'S LARA INSTITUTE OF TECHNOLOGY & SCIENCE

(AUTONOMOUS)

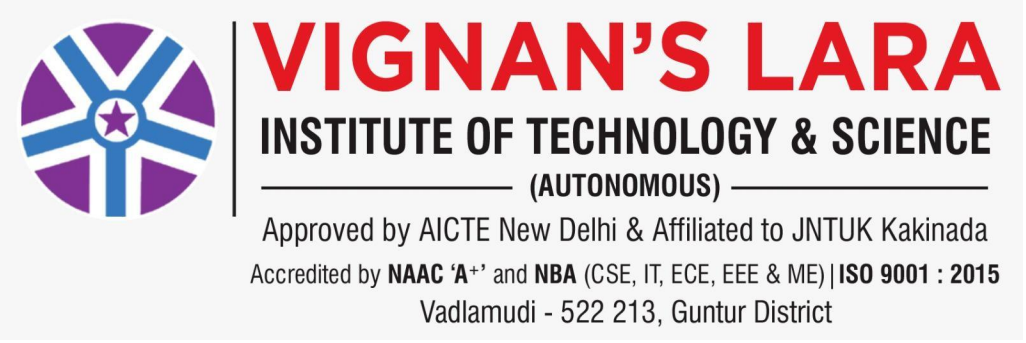
Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada

Accredited by **NAAC 'A+'** and **NBA** (CSE, IT, ECE, EEE & ME) | **ISO 9001 : 2015**

Vadlamudi - 522 213, Guntur District

2022-2026

DEPARTMENTOF ELECTRONICS AND COMMUNICATION ENGINEERING



2022 - 2026

CERTIFICATE

This is to certify that Project work entitled **"IMPLEMENTATION OF CRYPTO PROCESOR USING AES-RSA TECHNIQUES FOR SECURING VISUAL DATA INTEGRITY"** is a bonafide work done by **K. SRAVANI (22FE1A0456), K. RESHMA (22FE1A0472), P. JAYASREE (22FE1A0454), K. SYAMSUNDHAR (22FE1A0457)** under the guidance and submitted in partial fulfillment of requirement for the award of degree of **Bachelor of Technology** in **Electronics and Communication Engineering** by **Jawaharlal Nehru Technological University, Kakinada.**

Project Guide

Mr. CH. Raghunatha Babu, MTech, (Ph. D)

Associate Professor

Head of the Department

Dr. B. Harish, M. Tech, Ph .D

Professor

External Examiner

ACKNOWLEDGEMENT

We are most thankful to our parents who stood as pillars of motivation and for the way they influenced and molded our lives.

We are more thankful to our **c**hairman **Dr. LAVU RATHAIAH** who helped us to have technical incubation by providing the required infrastructure.

We are very much thankful to our principal **Dr. K. PHANEENDRA KUMAR** who extended a timely help at each and every step of our academic career.

We are very thankful to our Head of the department **Dr. B. HARISH** an amicable person who supported us very much and helped to a maximum extent and made this project successful.

We are very thankful to our beloved coordinators **Dr. P. AMALA VIJAYA SRI, Mrs. B. VIJAYA**, and **Mr**. **CH**. **RAGHUNATHA BABU** for inspiring all the way and arranging all the facilities and resources needed for project. Their efforts in this aspect are beyond the preview of the acknowledgement.

We are heartily thankful to our project guide **Mr. CH. RAGHUNATHA BABU,** Associate Professor, the person with vibrant knowledge and amicable by nature who laid a best guide line and for the efforts made by her to make the project a successful one.

Finally, we are thankful to each and every faculty member both technical and non technical, friends and all the persons who helped us directly or indirectly in making our project a successful one.

DECLARATION

We here by declare that the work describes in this project , entitled **“IMPLEMENTATIONOFCRYPTYO PROCESSOR USING AES-RSA TECHNIQUES FOR SECURING VISUAL DATA INTEGRITY ”**which is submitted by myself in partial fulfillment for the award of **Bachelor of Technology** in the Department of **Electronics and Communication Engineering** to the Vignan’s Lara Institute of Technology and Science affiliated to Jawaharlal Nehru Technological University Kakinada, Andhra Pradesh, is the result of work done by myself under the guidance of **Mr. CH. RAGHUNATHA BABU**, Associate Professor, in the Department ECE.

PROJECT ASSOCIATES

K.SRAVANI - 22FE1A0456

K.RESHMA - 22FE1A0472

P.JAYASREE - 22FE1A0454

K.SYAMSUNDUR - 22FE1A0457

Place: Vadlamudi

Date:

INDEX

LIST OF FIGURES

ABSTRACT

This paper presents the design and implementation of a high performance crypto processor integrating the Advanced Encryption Standard (AES) and Rivest Shamir Adleman (RSA) encryption algorithms using Verilog HDL. The proposed architecture combines the advantages of symmetric and asymmetric cryptographic techniques to achieve both high speed data processing and secure key management. AES is employed for efficient encryption and decryption of large data blocks due to its high throughput and low computational complexity, while RSA is utilized for secure key exchange and authentication, ensuring that only authorized users can access confidential information.

The crypto processor architecture is optimized for reduced power consumption and improved hardware efficiency, making it suitable for FPGA based implementations and embedded security applications. The design is modelled, simulated, and synthesized using industry standard tools, and performance metrics such as throughput, latency, and power utilization are analyzed. Simulation results demonstrate that the processor successfully performs encryption and decryption operations with high reliability and minimal resource usage.

Comparative analysis indicates that the proposed system achieves a balanced trade off between security strength and hardware efficiency. By integrating AES for bulk data encryption and RSA for secure key distribution, the system enhances overall data confidentiality, integrity, and authentication. The minimized power consumption and maximized throughput make the proposed crypto processor suitable for real time secure communication systems, cloud storage security, IoT devices, and large scale data protection applications.

CHAPTER 1

INTRODUCTION

1.1 What is a Crypto Processor

- It is the **brain of a computer** that controls and manages all operations.
- It **fetches, decodes, and executes instructions** to perform tasks like calculations and data handling.
- It works with **memory and input or output d e v i c e s** to make the whole system function.

1.2 Introduction to Crypto Processor

- A **Crypto processor** is a specialized hardware module designed to perform cryptographic operations such as encryption and decryption.
- It is used to secure data by converting plain text into cipher text and vice versa.
- It provides hardware-based security, which is faster and more secure than software- based encryption
- It supports cryptographic algorithms like **Advanced Encryption Standard (AES)** and **RSA** .
- It is widely used in secure communication systems, banking applications, IoT devices, and cloud security.
- A crypto processor reduces power consumption by optimizing encryption operations at the hardware level

1.3 Why only this?

A crypto processor is a specialized hardware unit designed to perform cryptographic operations such as encryption, decryption, key generation, and authentication. Unlike software- based encryption, a crypto processor executes security algorithms directly in hardware, which makes the system faster, more secure, and power-efficient. It supports standard cryptographic algorithms such as Advanced Encryption Standard (AES) for high-speed data encryption and RSA for secure key exchange and authentication. The main purpose of a crypto processor is to ensure data confidentiality, integrity, and secure access by authorized users. It is widely used in applications such as secure communication systems, banking networks. In addition, crypto processors are optimized for efficient resource utilization, making them suitable for FPGA and ASIC implementations in embedded systems. They are widely used in secure communication networks, financial transaction systems.

By offloading cryptographic tasks from the main processor, they improve overall system performance while minimizing power consumption and latency. Their ability to process large volumes of data securely and in real time makes them essential components in modern cybersecurity architectures and next-generation digital systems.

Furthermore, a crypto processor enhances system-level security by incorporating additional features such as secure key storage, random number generation, and tamper detection mechanisms. These features prevent unauthorized access and protect cryptographic keys from physical and logical attacks. In many designs, the processor includes hardware-based protection techniques such as side-channel attack resistance, secure boot support, and protected memory regions. By embedding these security mechanisms directly into hardware, the overall trustworthiness of the system is significantly improved.

Moreover, modern crypto processors are designed to be scalable and configurable, allowing support for multiple cryptographic standards and varying key sizes depending on application requirements. For example, algorithms such as Advanced Encryption Standard can be configured with different key lengths to enhance security levels, while RSA can be implemented with large key sizes for stronger authentication. This flexibility makes crypto processors suitable for emerging technologies such as 5G communication, blockchain systems, smart cards, and Internet of Things (IoT) devices. As cyber threats continue to evolve, crypto processors play a vital role in ensuring secure, efficient, and reliable digital communication across various platforms.

CHAPTER 2

LITERATURE SURVEY

Crypto processors highlights hardware implementations of cryptographic algorithms like AES and RSA to achieve faster encryption and decryption compared to software-only solutions. Previous works emphasize reducing power consumption and improving throughput in FPGA and ASIC-based designs for real-time secure applications.

2.1 Fundamental Principles of Crypto Processor

The fundamental principles of a crypto processor are based on ensuring secure, efficient, and reliable data protection within a hardware system. A crypto processor is designed to maintain confidentiality by encrypting sensitive information using strong cryptographic algorithms such as Advanced Encryption Standard and RSA. It also ensures data integrity by preventing unauthorized modification and supports authentication mechanisms to verify the identity of users or devices. Secure key management is another core principle, where cryptographic keys are generated, stored, and processed internally within protected hardware modules to avoid exposure.

2.2 Implementation Strategies

The implementation of a crypto processor requires a structured hardware design approach to achieve high security, speed, and efficiency. One common strategy is the integration of symmetric and asymmetric cryptographic algorithms, such as Advanced Encryption Standard for fast bulk data encryption and RSA for secure key exchange and authentication. This hybrid approach ensures both high throughput and strong security. The architecture is typically divided into modular blocks, including encryption/decryption units, key generation modules, control logic, and memory interfaces, allowing easier design, testing, and scalability.

2.3 Performance Analysis

Performance analysis of crypto processor evaluates how efficiently it performs encryption and decryption operations while maintaining strong security. The key performance metrics include **throughput, latency, power consumption, area utilization, and resource efficiency**. Throughput measures the amount of data processed per unit time, which is significantly high when using hardware-based implementations of algorithms such as Advanced Encryption Standard for bulk data encryption. Latency refers to the time taken to complete a single encryption or decryption operation, which is minimized through pipelining and parallel processing techniques. Power consumption is another critical factor, especially in FPGA and embedded implementations, where optimized architecture.

TITLE: COMPARATIVE ANALYSIS OF AES AND AES – RSA HYBRID TECHNIQUES FOR SECURING VISUAL DATA INTEGRITY [1]

AUTHOR: ELUMALAIVASAN

Comparative Analysis of AES and AES – RSA Hybrid Techniques for Securing Visual Data Integrity" address the image encryption using the RSA Key Encryption technique used in Python for offering easy data Transmission. Easier to key management for un authorized Raghu users. Our proposed system will access for large files. Here small data access will be available for large data it doesn't support .The techniques are sequential and this will be have limited access for users. The power consumption is very high.

TITLE: A REVIEW ON IMAGE ENCRYPTION AND DECRYPTION [2]

AUTHOR: BHANDARI AND MUNDARGI

A Review on Image Encryption and Decryption, exploring the evolution of cryptography from ancient civilizations to modern techniques. They extensively analyze symmetric encryption algorithms like DES and AES, along with asymmetric methods such as RSA and ECC (Elliptic curve cryptography), emphasizes their role in communication security. The Elliptic Curve Cryptography is a modern public key cryptographic technique that provides strong security using the mathematics of elliptic curves. it is based on the elliptic curve equation and relies on the difficulty of solving the ECC problem. In ECC a user generates a private key and derives a public key. Due to this it offers the same level security like RSA but much smaller key size files So that it will access for only small files. The review extends to key management strategies and practical implementations across various sectors, highlighting ongoing challenges and emerging threats in cryptographic landscapes.

TITLE: STRENGTHENING RSA ENCRYPTION VIA HYBRID CRYPTO SYSTEM AND KEY MANAGEMENT [4]

AUTHOR: AMIT PRASAD AND VINOTH KUMAR M

An enhanced hybrid cryptographic framework to strengthen traditional RSA-based encryption systems. Although RSA offers safe public-key encryption, it has performance issues and is vulnerable to timing attacks and chosen-ciphertext attacks. The suggested model combines RSA for safe session key exchange and AES for effective bulk data encryption to solve these problems. Robust key lifecycle management guarantees safe key generation, rotation, and disposal, while the application of Optimal Asymmetric Encryption Padding (OAEP) increases resistance against adaptive chosen ciphertext attacks. But it is an Asymmetric Encryption so the data will be hacked easily by un authorizers.

TITLE : COLOUR IMAGE ENCRYPTION USING AES AND RSA [6]

AUTHOR: JASWANTH REDDY ,PRIYADARSHINI

To implement robust information security measures for color images through their research on "Color Image Encryption using AES and RSA". Image representation involves treating images as arrays, accommodating grayscale and color images. Their study showcases the efficacy of AES and RSA encryption for color image security, validated through MATLAB evaluations based on specified criteria. It can be access for low files only rather than our proposed system is very efficient.

TITLE: IMAGE ENCRYPTION AND DECRYPTION USING AES ALGORITHMS [8]

AUTHOR: PADATE AND PATEL

The critical need for data security in communication systems through their paper "Image Encryption and Decryption Using AES Algorithm." They focus on the implementation of AES for encryption and decryption, leveraging its effectiveness in securing communication channels. The paper highlights AES Key Expansion and its dynamic 128 bit keys, positioning AES as a versatile algorithm adaptable to diverse environments. They use only 128 bits of data and there is no security for data transfer . some un authorizers will access the data . that will be crucial for transferring the data.

2.4 Recent Advances and Hybrid Approaches

RISC-V crypto processors has focused on advanced hybrid approaches that combine multiple cryptographic techniques to achieve both high performance and strong security. These approaches typically integrate symmetric algorithms like Advanced Encryption Standard for fast bulk data encryption with asymmetric algorithms such as RSA for secure key exchange and authentication. Some modern designs also incorporate post-quantum cryptography, such as lattice-based schemes, alongside classical algorithms to protect against emerging quantum attacks.

2.5 Conclusion of Literature Survey

Crypto processors highlights a clear trend toward combining multiple cryptographic techniques to achieve both security and performance efficiency. Research shows that integrating symmetric algorithms like Advanced Encryption Standard with asymmetric methods such as RSA enhances data confidentiality, secure key management, and authentication. Recent advancements emphasize hardware.

CHAPTER 3 SOFTWARE REQUIREMENTS

3.1 Overview of Xilinx Software

Xilinx VIVADO is a comprehensive and advanced integrated development environment (IDE) and design toolset primarily used for designing and programming Xilinx FPGAs (Field-Programmable Gate Arrays) and SoCs (System on Chips).

3.1.1 About Software:

FPGA and SoC Development: VIVADO is a versatile software tool that is primarily used for the design, analysis, and programming of Xilinx FPGAs and SoCs. It supports a wide range of Xilinx devices, from entry-level FPGAs to high- end, high- performance chips. VIVADO provides support for high-level design entry, allowing engineers and developers to use languages like VHDL, Verilog, and high-level programming languages such as C, C++, and OpenCL to create hardware designs. The toolset includes a large library of pre-designed Intellectual Property (IP)cores that can be easily integrated into your FPGA or SoC design. This simplifies the design process and reduces development time. VIVADO performs logic synthesis to convert high-level descriptions of your design into the lower-level logic gates that will be programmed into the FPGA or So C. It optimizes the design for performance, area, and power consumption.

3.1.2 Implementation, Placing and Routing:

VIVADO provides tools for physical implementation and place-and- route, which determine the exact arrangement of logic elements and interconnections on the FPGA or So C. This step is crucial for optimizing the design for target hardware. The software includes timing analysis tools to ensure that your design meets the required performance specifications, and that signals propagate correctly through the device. VIVADO offers debugging and verification tools, including hardware debugging through JTAG, simulation capabilities, and interfaces with third-party simulation tools. We can use VIVADO to generate bit streams that configure the FPGA.

3.2 Working with the VIVADO IDE:

The VIVADO Integrated Design Environment (IDE) can be used in both Project Mode and Non Project Mode. The VIVADO IDE provides an interface to assemble, implement, and validate your design and IP. Opening a design loads the current design netlist, applies design constraints, and fits the design onto the target device. The VIVADO IDE allows you to visualize and interact with the design as shown in the following figure.

Launching The VIVADO IDE On Windows

Select Start → All Programs → Xilinx Design Tools → VIVADO → VIVADO.

Note: You can also double-click the VIVADO IDE shortcut icon on your desktop.

Fig 3.1: VIVADO IDE Desktop Icon

3.3 Creating Projects:

The VIVADO Design Suite supports different types of projects for different design purposes. For example, you can create a project with RTL sources or synthesized. You can also create empty I/O planning projects to enable device exploration and early pin planning. The Vivado IDE only displays commands relevant to the selected project type .In the VIVADO IDE, the Create Project wizard walks you through the process of creating a project. The wizard enables you to define the project, including the project name, the location in which to store the project, the project type (for example, RTL, netlist, and so forth), and the target part. You can add different types of sources, such as RTL, IP, Block designs, XDC or SDC constraints, simulation test benches, DSP modules from System Generator as IP, or VIVADO High Level Synthesis (HLS), and design documentation. When you select sources, you can determine whether to reference the source in its original location or to copy the source into the project directory.

Project wizard walks you through the process of creating a project. The wizard enables you to define the project, including the project name, the location in which to store the project, the project type (for example, RTL, netlist, and so forth), and the target part. You can add different types of sources, such as RTL, IP, Block designs, XDC or SDC constraints, simulation test benches, DSP modules from System Generator as IP, or VIVADO High Level Synthesis (HLS), and design documentation. When you select sources, you can determine whether to reference the source in its original location or to copy the source into the project directory. The VIVADO Design Suite tracks the time and date stamp of each file and report status. If files are modified, you are alerted to out-of-date source or design status. For more information, see this link in the VIVADO Design Suite User Guide: System-Level Design Entry.

3.3.1 Different Types of Projects:

The VIVADO Design Suite allows for different design entry points depending on your source file types and design tasks. Following are the different types of projects you can use to facilitate those tasks:

- **RTL Project:** You can add RTL source files and constraints, configure IP with the VIVADO IP catalog, create IP subsystems with the VIVADO IP integrator, synthesize and implement the design, and perform design planning and analysis.
- **Post-Synthesis Project:** You can import third-party netlists, implement the design, and perform design planning and analysis.
- **I/O Planning Project:** You can create an empty project for use with early I/O planning and device exploration prior to having RTL sources.
- **Imported Project:** You can import existing project sources from the ISE Design Suite, Xilinx Synthesis Technology (XST), or Synopsys.
- **Example Project:** You can explore several example projects, including example Zynq - 7000 SoC or Micro Blaze™ embedded designs with available Xilinx evaluation boards. :
- **Data F x:** You can dynamically reconfigure an operating FPGA design by loading a partial bitstream file to modify reconfigurable regions of the device.

3.4 Understanding The Flow Navigator:

The Flow Navigator (shown in the following figure) provides control over the major design process tasks, such as project configuration, synthesis, implementation, and bitstream generation. The commands and options available in the Flow Navigator depend on the status of the design. Unavailable steps are grayed out until required design tasks are completed.

Fig 3.2: Flow Navigator

The Flow Navigator (shown in the following figure) differs when working with projects created with third-party netlists. For example, system-level design entry, IP, and synthesis options are not available. Flow Navigator in Xilinx Vivado is a left pane, push button interface that controls the core design flow, enabling users to manage projects, run synthesis and implementation, generate bitstreams, and manage IP. It streamlines tasks from design entry to hardware programming by visually organizing steps and graying out unavailable actions until necessary preceding steps are completed. It serves as a standardized interface providing easy access to all essential tools and commands required to guide a design from, Synthesis, Implementation, and Program or Debug.

Flow Navigator for Third-Party Netlist Project. As the design tasks are complete, you can analyze the RTL by performing Linting on it and specifying the waive-specific violations in the current design. After cleaning up the design, you can open the resulting designs to analyze the results and apply constraints. In the Flow Navigator When you open a design, the Flow Navigator shows a set of commonly used commands for the applicable phase of the design flow. Selecting any of these commands in the Flow Navigator opens the design, if it is not already opened, and performs the operation. For example, the following figure shows the commands related to synthesis.

The VIVADO IDE Sources window (shown in the following figure) provides automated source file management. The window has several views to display the sources using different methods. When you open or modify a project, the Sources window updates the status of the project sources. A quick compilation of the design source files is performed and the sources appear in the Compile Order view of the Sources window in the order they will be compiled by the downstream tools. Any potential issues with the compilation of the RTL hierarchy are shown as well as reported in the Message window.

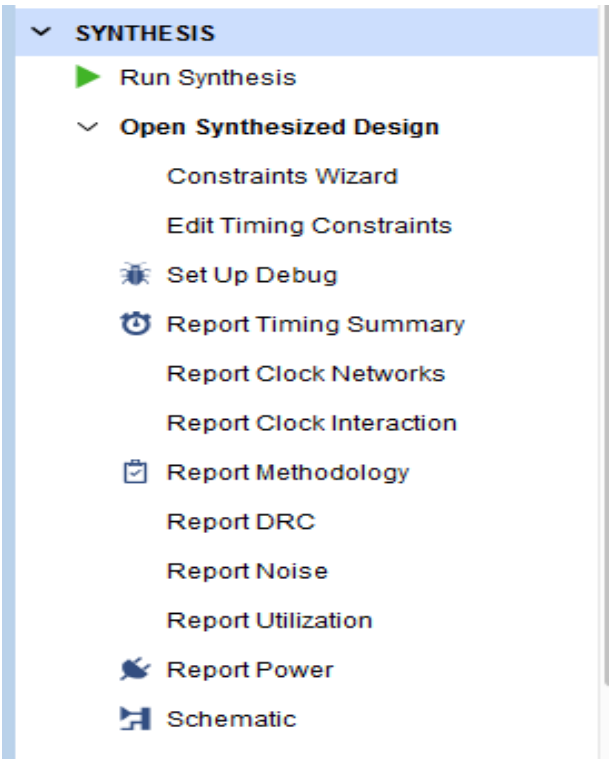


Fig 3.3: Synthesis Section in the Flow Navigator

3.5 Automated Hierarchical Source File Compilation and Management:

The VIVADO IDE Sources window (shown in the following figure) provides automated source file management. The window has several views to display the sources using different methods. When you open or modify a project, the Sources window updates the status of the project sources. A quick compilation of the design source files is performed and the sources appear in the Compile Order view of the Sources window in the order they will be compiled by the downstream tools. Any potential issues with the compilation of the RTL hierarchy are shown as well as reported in the Message window. For more information on sources, see this link in the VIVADO Design Suit.

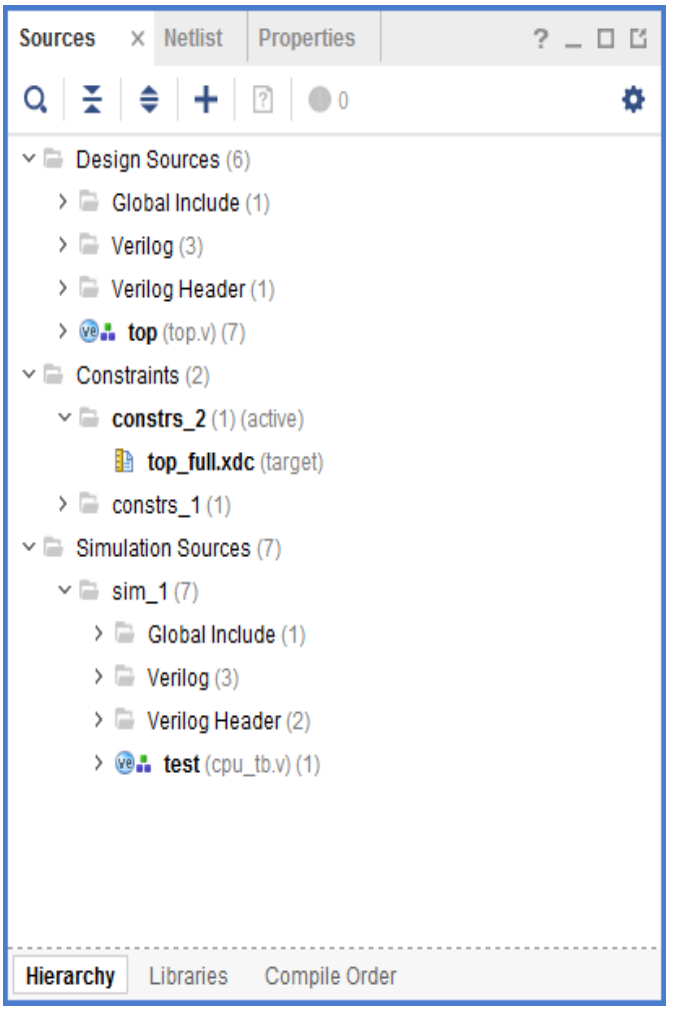


Fig 3.4: Hierarchical Sources View Window

3.6 Simulation Flow:

The VIVADO Design Suite supports both integrated simulation, which allows you to run the simulator from within the VIVADO IDE, and batch simulation, which allows you to generate a script from the VIVADO tools to run simulation on an external verification environment.

3.6.1. Integrated Simulation

The VIVADO IDE provides full integration with the VIVADO simulator, and all supported third-party simulators. In this flow, the simulator is called from within the VIVADO IDE, and you can compile and simulate the design easily with a push of a button, or with the launch simulation T + ctrl command.

3.6.2. Running Logic Synthesis And Implementation

- **Logic Synthesis**

VIVADO synthesis enables you to configure, launch, and monitor synthesis runs. The VIVADO IDE displays the synthesis results and creates report files. You can select synthesis warnings and errors from the Messages window to highlight the logic in the RTL source files.

You can launch multiple synthesis runs concurrently or serially. On a Linux system, you can launch runs locally or on remote servers. With multiple synthesis runs, VIVADO synthesis creates multiple netlists that are stored with the VIVADO Design Suite project. You can open different versions of the synthesized netlist in the VIVADO IDE to perform device and design analysis. You can also create constraints for I/O pin planning, timing, floor planning, and implementation. The most comprehensive list of DRCs is available after a synthesized netlist is produced, when clock and clock logic are available for analysis and placement.

- **Implementation**

VIVADO implementation enables you to configure, launch, and monitor implementation runs. You can experiment with different implementation options and create your own reusable strategies for implementation runs. For example, you can create strategies for quick run times, improved system performance, or area optimization. As the runs complete, implementation run results display and report files are available.

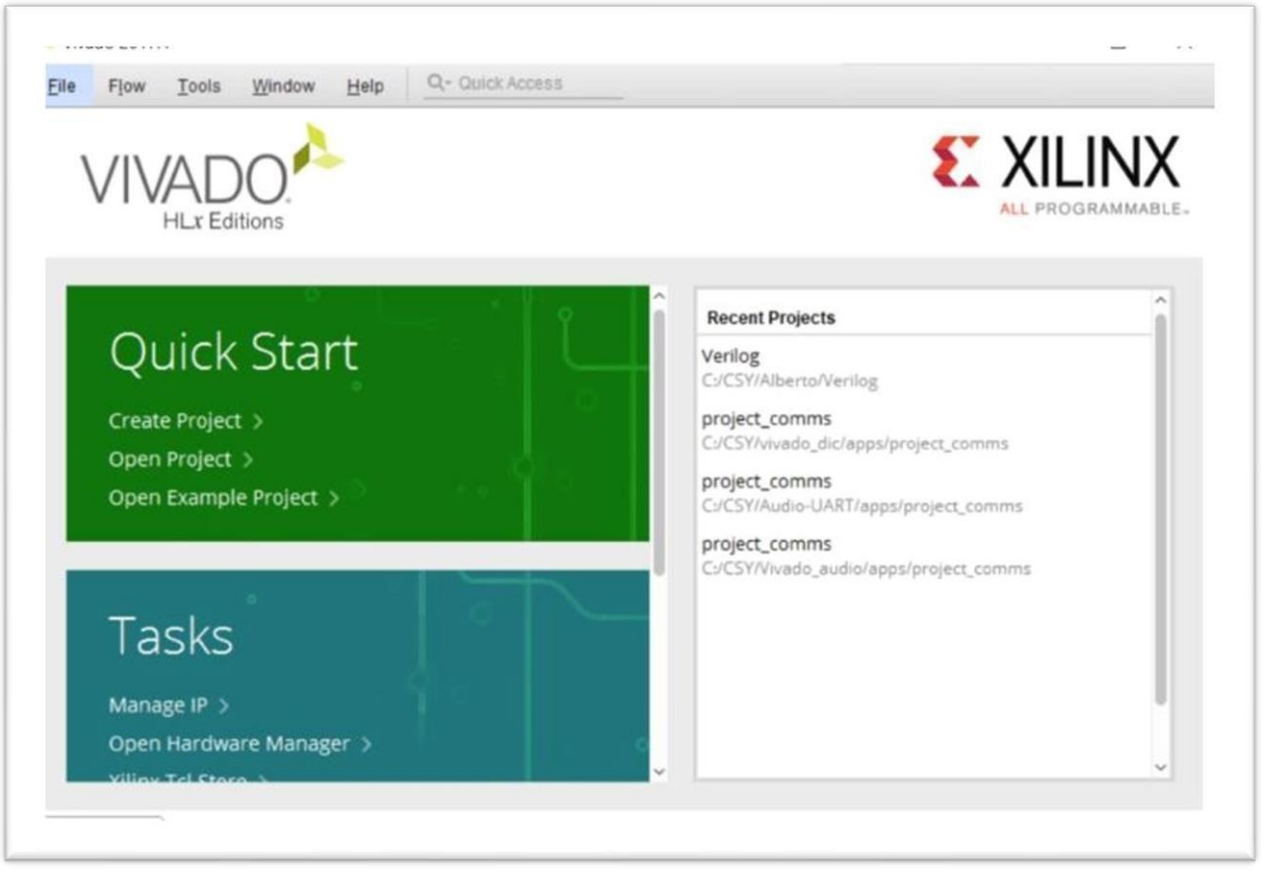


Fig 3.5: Interface of Xilinx Vivado

3.7 Applications of VIVADO Software:

VIVADO is primarily used for the development of FPGA and SoC designs. It provides a robust platform for hardware design, simulation, synthesis, and implementation, making it a go-to tool for creating custom digital logic circuits and embedded systems. VIVADO is extensively used in DSP applications, where high- performance signal processing is required. It can accelerate the design and implementation of algorithms on FPGAs for applications like image and speech processing.

3.7.1 .Aerospace and Defense:

In industries such as aerospace and defense, VIVADO is used to develop high- reliability systems. FPGAs offer flexibility and can be rapidly reconfigured, making them suitable for applications like radar, communications, and secure data processing. VIVADO also plays a vital role in communication systems, where FPGAs are used to

implement protocol and interface converters, high-speed data processing, and encryption/decryption modules. In the automotive industry, FPGAs and SoCs designed with VIVADO are used for applications like advanced driver assistance systems (ADAS), infotainment, and in-vehicle networking.

3.7.2 Industrial Automation:

VIVADO is employed in industrial automation for developing control systems, data acquisition, and processing solutions. It offers real-time performance and customization options for various industrial applications. VIVADO is widely used in academia for research, teaching, and hands-on learning in digital design, FPGA programming, and embedded systems. It helps students and researchers gain practical experience in FPGA technology. As IoT and edge computing applications grow. VIVADO is used to design FPGA and SoC solutions that can process data at the network edge efficiently, enabling real-time analytics and decision-making.

CHAPTER 4

CRYPTO PROCESSOR IMPLEMENTATION

4.1 Introduction

1. RISC-V:

- A crypto processor is a specialized hardware unit designed to perform cryptographic operations such as encryption, decryption, key generation, and authentication. Unlike software-based encryption, a crypto processor executes security algorithms directly in hardware, which makes the system faster, more secure, and power efficient. It supports standard cryptographic algorithms such as Advanced Encryption Standard (AES) for high-speed data encryption and RSA for secure key exchange and authentication.

2. Early Processor:

- Cryptographic operations were primarily performed in software on general purpose microprocessors or computers. Encryption and decryption tasks relied on the CPU to execute algorithms like DES (Data Encryption Standard) or early versions of RSA.
- It was slow, consumed a lot of processing power, and could not efficiently handle large volumes of data or real-time security requirements.

3. Crypto Processor :

- Crypto processor uses a modular and open-source RISC architecture, allowing customizable instruction sets for optimized performance and flexibility in various applications.
- Limitations: Limited software ecosystem and toolchain support compared to mature architectures which can slow adoption in mainstream commercial systems.

4.2 Need for Crypto processor

Crypto processors were specialized hardware units developed primarily to perform cryptographic operations faster than software-based implementations. These processors focused on implementing fundamental symmetric encryption algorithms like DES (Data Encryption Standard) and simple asymmetric algorithms such as RSA with small key sizes. The architecture of early processors was basic, often consisting of dedicated encryption/decryption units, simple control logics.

4.3 Introduction to the Crypto Processor

The y was introduced to address the limitations of the RCA by improving speed through parallel processing. Here's a deeper dive into its architecture and operation: How the CSA Works

1. **Open-Source Architecture:** Crypto is a free and open standard instruction set architecture (ISA) that allows anyone to design, implement, and modify processors.
2. **Reduced Instruction Set Computing:** It uses a simplified set of instructions to improve performance and efficiency in hardware design**.**
3. **Scalable and Flexible:** Suitable for a wide range of applications, from microcontrollers and IoT devices to high-performance processors.
4. **Encourages Innovation:** Its modular and extensible design allows customization and avoids vendor lock-in, fostering research and industrial development.

4.4 Detailed

- **Program Counter (PC) Management:** Maintains the address of the next instruction to fetch from memory.
- **Instruction Memory Access:** by Retrieves the instruction stored at the address pointed the PC.
- **PC Increment Logic:** Updates the PC to point to the next sequential instruction or branch target.
- **Instruction Buffering:** Temporarily holds the fetched instruction before passing it to the decode stage.
- **Instruction Parsing:** Interprets opcode, source/destination registers, and immediate values from the fetched instruction.
- **Register File Access:** Reads the values of source registers required for execution.
- **Immediate Value Processing:** Extracts and sign-extends or zero-extends immediate operands as needed.

4.5 Architecture of AES Using RSA :

The architecture of AES using RSA is based on a hybrid cryptographic model that combines the strengths of symmetric and asymmetric encryption techniques. In this system, the Advanced Encryption Standard is used to encrypt the main data, such as an image or file, while the RSA algorithm is used to securely encrypt the AES secret key. This approach ensures both high-speed data encryption and secure key distribution. AES is selected for encrypting the bulk data because it operates efficiently on 128-bit blocks and provides strong confusion and diffusion through multiple rounds of transformation.

On the sender side, a random AES key is generated and used to encrypt the input image through Sub Bytes, Shift Rows, Mix Columns, and Add Round Key operations. The output is an encrypted image that appears random and unreadable. Since AES is a symmetric algorithm, the secret key must be securely transmitted to the receiver. To solve this problem, RSA encryption is applied to the AES key using the receiver's public key. The encrypted AES key and the encrypted image are then transmitted together over the communication channel.

On the receiver side, the RSA private key is used to decrypt and recover the original AES secret key. Once the AES key is obtained, it is applied to decrypt the encrypted image and restore the original data. This architecture ensures confidentiality, secure key exchange, and resistance to key interception attacks. By combining the efficiency of AES with the strong mathematical security of RSA, the hybrid architecture provides enhanced protection for secure image transmission and storage systems.

At the receiver side, RSA decryption is first performed to retrieve the original AES key. The recovered key is then used in the AES decryption module, which applies inverse operations such as Shift Rows, Sub Bytes, Mix Columns, and Add Round Key to reconstruct the original image. This sequential architecture ensures that even if the encrypted image is intercepted, it cannot be decrypted without access to both RSA private key and AES key.

Overall, the AES-RSA architecture provides a balanced trade-off between speed and security. AES ensures fast processing of large visual data, while RSA guarantees secure key exchange and authentication.

4.6 Comparative Analysis of AES and RSA :

AES (Advanced Encryption Standard) and RSA (Rivest-Shamir-Adleman) are two of the most widely used encryption algorithms in modern cryptography, yet they are fundamentally different in design, purpose, and operation.

AES is a **symmetric key encryption algorithm**, which means that the same secret key is used for both encryption and decryption, whereas RSA is an **asymmetric encryption algorithm**, using a pair of keys: a public key for encryption and a private key for decryption. The symmetry in AES allows it to perform encryption extremely efficiently, as the same key can be applied repeatedly across data blocks, making it suitable for encrypting large amounts of data quickly. RSA's asymmetry, on the other hand, relies on the mathematical complexity of factoring very large integers, which requires significantly more computation and longer key sizes, typically 2048 or 4096 bits, to achieve comparable security.

AES operates on fixed-size blocks of data, usually 128 bits, and supports key lengths of 128, 192, or 256 bits. This design allows AES to handle high-throughput encryption, making it highly suitable for applications such as file encryption, image and video data protection, and secure communication over networks. RSA does not operate on blocks in the same sense; it encrypts data in smaller segments due to the limitations of its mathematical operations, making it impractical for encrypting large datasets directly. Instead, RSA is primarily used for **key exchange, digital signatures, and authentication**, where encrypting small amounts of data securely is more important than speed.

AES uses substitution-permutation networks with multiple rounds of processing, where each round applies operations like substitution, permutation, and mixing with round keys derived from the main key. This provides a high degree of diffusion and confusion, ensuring that small changes in plaintext or key produce large changes in ciphertext. RSA relies on modular exponentiation of very large numbers and the difficulty of factoring their product of two large primes. Its security is grounded in number theory, and while it is mathematically robust, it is inherently slower and more resource-intensive than AES.

Performance-wise, AES is **significantly faster than RSA**, especially for large datasets. Hardware implementations of AES can be parallelized, pipelined . which can limit its use in resource-constrained environments. For applications requiring secure authentication, digital signatures, or certificate validation, RSA remains indispensable, while AES alone cannot provide these asymmetric functionalities.

RSA's security is strong under classical computation, but it relies on the difficulty of factoring large integers, which may be vulnerable to quantum computing in the future through Shor's algorithm. Consequently, AES is considered more **resilient to future quantum attacks** than RSA, which has prompted research into post-quantum cryptography alternatives for RSA-like schemes.

Encryption of high-bandwidth data streams. RSA, due to its reliance on large integer arithmetic and modular exponentiation, is computationally expensive, making it unsuitable for bulk data encryption. This is why most modern cryptographic protocols adopt a **hybrid approach**, where AES encrypts the bulk data efficiently and RSA encrypts the AES key for secure key exchange.

From a security standpoint, AES with a 256 bit key is considered practically unbreakable with current technology. Brute-force attacks on AES-256 are computationally infeasible even with large-scale supercomputers. RSA's security is strong under classical computation, but it relies on the difficulty of factoring large integers, which may be vulnerable to quantum computing in the future through Shor's algorithm. Consequently, AES is considered more **resilient to future quantum attacks** than RSA, which has prompted research into post-quantum cryptography alternatives for RSA-like schemes.

AES also has advantages in **resource efficiency**. It requires minimal memory and computational resources, making it ideal for embedded systems, IoT devices, and mobile hardware where processing power and energy consumption are critical. RSA's operations are more demanding, requiring larger memory and longer computation times, which can limit its use in resource-constrained environments. For applications requiring secure authentication, digital signatures, or certificate validation, RSA remains indispensable, while AES alone cannot provide these asymmetric functionalities.

AES is deterministic in the sense that encryption with the same key and plaintext always produces the same ciphertext unless additional mechanisms like initialization vectors (IVs) are used for modes such as CBC or CTR.

RSA inherently involves randomness in certain implementations for padding schemes like OAEP, which enhances security against chosen plaintext attacks. This difference affects how the two algorithms are used in protocols: AES ensures high-speed, predictable encryption, while RSA adds security features necessary for key management and authentication.

In practical applications, AES is used extensively in HTTPS, VPNs, cloud storage encryption, file encryption software, and secure messaging platforms, whereas RSA is widely used for TLS handshakes, digital certificates, email encryption, and secure key distribution.

The combination of AES and RSA in hybrid cryptosystems allows systems to leverage AES's efficiency for data encryption while using RSA to securely exchange AES keys , ensuring that even if communication channel.

AES's simplicity and suitability for hardware acceleration mean that it can achieve encryption speeds of several gigabits per second in FPGA or dedicated hardware, which is critical for streaming applications, video conferencing, or high-speed network encryption.

RSA, in contrast, typically operates at kilobits per second to a few megabits per second depending on key size and implementation, making it unsuitable for encrypting large data streams directly. This performance gap reinforces why hybrid encryption is the standard for secure communications today.

Key management is another area of difference. In AES, the same key must be shared securely between parties before communication, necessitating a secure channel or key exchange mechanism, such as RSA. RSA inherently solves the key distribution problem because its public key can be shared openly while the private key remains secret. This property makes RSA essential for **establishing trust and distributing AES keys securely**. AES cannot provide authentication or verify sender identity on its own, whereas RSA can, through digital signatures.

AES's resistance to cryptanalysis is well-studied, with no practical attacks on full 128 , 192 , or 256 bit versions. Weaknesses may appear in poorly implemented key schedules, side-channel attacks, or incorrect padding, but these are generally implementation flaws rather than algorithmic vulnerabilities. RSA's security depends on key length and implementation.

Vulnerabilities can arise from small key sizes, poor random number generation for primes, or timing attacks in hardware and software implementations. Proper padding schemes like OAEP are critical to prevent chosen- ciphertext attacks in RSA.

In terms of flexibility, AES can operate in multiple modes (ECB, CBC, CFB, OFB, CTR, GCM), allowing encryption to be tailored for specific performance, parallelization, and security requirements. GCM mode, for example, adds **authenticated encryption**, combining confidentiality and integrity in a single operation.

RSA's flexibility is mostly limited to key size, padding schemes, and signature algorithms. It is not suitable for bulk encryption but is highly versatile for digital signatures, authentication, and establishing secure communication channels.

Hybrid cryptosystems that combine AES and RSA provide the **best of both worlds**: AES ensures fast, efficient encryption of large data blocks, while RSA provides asymmetric security for key exchange and digital authentication. This combination underlies protocols like **TLS SSL**, which secures web traffic .

Without RSA, securely distributing AES keys would be difficult over untrusted channels.

In hardware design, AES benefits from **parallel processing and pipeline architectures**, which allow multiple blocks to be encrypted simultaneously, significantly improving throughput. RSA's arithmetic operations, especially modular exponentiation on large integers, are harder to parallelize efficiently, leading to higher latency. Software implementations reflect similar patterns: AES can handle streaming data with minimal CPU overhead, while RSA consumes significant CPU cycles and memory.

Energy efficiency is another consideration. AES's simple operations XOR, substitution, and permutation consume less power compared to RSA's large integer multiplications and modular exponentiations.

For battery-powered devices, embedded systems, and IoT applications, AES is preferable for encrypting data, while RSA is used sparingly for secure key exchange or authentication.

From a key-size perspective, AES achieves strong security with relatively short keys (256 bits), whereas RSA requires extremely large keys (2048–4096 bits) to achieve equivalent security levels. This size difference affects storage, memory, and computational requirements, making AES more lightweight for repeated encryption tasks.

AES also scales well with modern computing architectures, including GPUs and multicore CPUs, which can exploit its parallelizable structure to achieve extremely high throughput. RSA, being sequential in nature due to modular exponentiation, does not benefit as much from parallelization, further emphasizing its role as a key management tool rather than a bulk encryption mechanism.

AES is widely used in **data-at-rest encryption**, such as encrypting files on hard drives, SSDs, or cloud storage, as well as **data-in-transit encryption** over secure network channels. RSA is critical in **data in transit security** for establishing secure channels, encrypting session keys, and verifying digital signatures. Together, they provide end to end security for communications and storage.

Side channel attacks are a consideration for both algorithms. AES can be vulnerable to timing attacks, power analysis, or cache attacks if poorly implemented, especially in embedded devices. RSA is also vulnerable to timing attacks, fault attacks, and improper padding usage.

Their respective strengths and weaknesses define the architecture of secure systems today, demonstrating how symmetric and asymmetric cryptography can work in tandem to achieve comprehensive, reliable, and scalable security. AES handles bulk data efficiently, RSA handles key security and authentication, and together they form the backbone of modern encryption protocols that protect sensitive information across the globe.

Their widespread adoption reflects their complementary strengths and proven reliability in diverse applications.

AES is deterministic by default but can be combined with **randomized initialization vectors** to produce unique ciphertexts for identical plaintexts, which enhances security against certain attacks. RSA incorporates probabilistic padding schemes like OAEP to ensure security against chosen-ciphertext attacks. These differences influence protocol design and how each algorithm is applied in practice.AES also supports authenticated encryption modes that provide **both confidentiality and integrity**, whereas RSA alone cannot provide data integrity without additional cryptographic constructs like digital signatures or hash functions.

AES is well suited for **streaming and real-time applications**, such as video conferencing, live media streaming, or encrypted voice communication. RSA is typically used at the beginning of a session to securely exchange keys and establish trust. Once the session key is securely transmitted, AES handles all subsequent data efficiently.

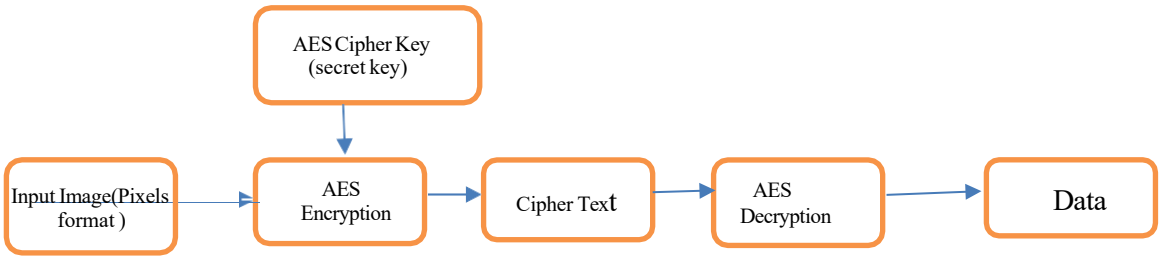
AES and RSA are complementary cryptographic tools: AES provides **fast, efficient, and secure encryption for large datasets**, while RSA provides **secure key exchange, digital authentication, and asymmetric security**.

AES excels in speed, parallelization, low resource usage, and bulk data encryption, while RSA excels in enabling secure communication without prior key sharing and providing authentication via digital signatures. Their combined use in **hybrid encryption systems** such as those in TLS/SSL, PGP, and secure cloud services ensures both **high performance and strong security**, making them indispensable in modern cryptographic applications.

The distinctions between AES and RSA highlight why modern security protocols rarely rely on a single encryption algorithm. AES handles the heavy lifting of encrypting large amounts of data efficiently .while RSA guarantees that the keys used in AES are transmitted securely and cannot be intercepted by unauthorized parties. Together, they create a system where **confidentiality, integrity, and authenticity** are all ensured without compromising performance.

Overall, AES and RSA complement each other perfectly: AES provides high-speed, efficient, and secure encryption of data, while RSA provides key security, authentication, and asymmetric encryption capabilities. Their respective strengths and weaknesses define the architecture of secure systems today, demonstrating how symmetric and asymmetric cryptography can work in tandem to achieve comprehensive, reliable, and scalable security

BLOCK DIAGRAM OF AES :



Fig(a) : Block Diagram of AES Encryption and Decryption

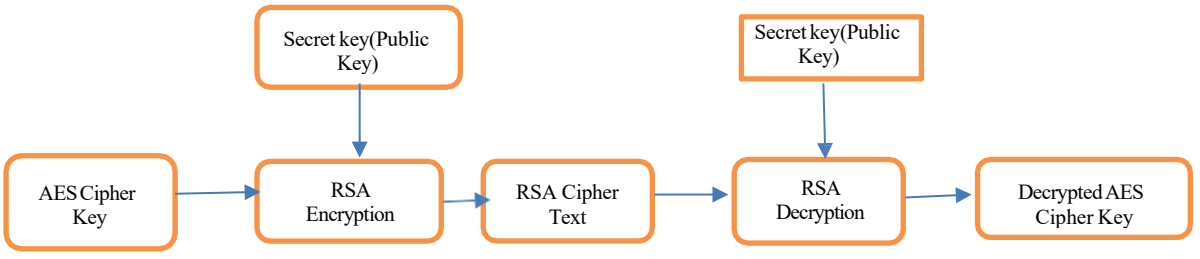
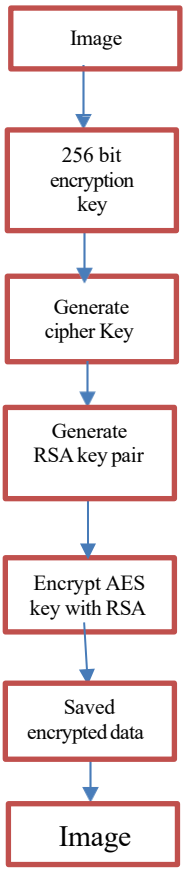


Fig (b) : Block Diagram of AES Key Encryption and Decryption

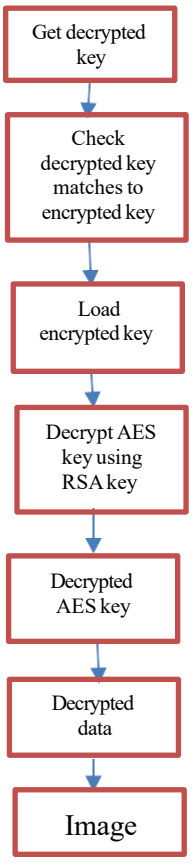
Fig 4.2 : Block Diagram of AES Encryption and Decryption

The **Advanced Encryption Standard (AES)** architecture is based on a symmetric key substitution permutation network designed to provide high security with efficient hardware implementation. AES operates on a fixed 128-bit data block, which is arranged internally as a 4×4 byte state matrix. The encryption process begins with an initial Add Round Key operation, where the plaintext is XORed with the secret key. This is followed by multiple encryption rounds 10, 12, or 14 rounds depending on the key size of 128, 192, or 256 bits. Each round consists of four core transformations: Sub Bytes, which performs non-linear byte substitution using an S-box; Shift Rows, which cyclically shifts the rows of the state matrix to achieve diffusion; Mix Columns, which applies Galois Field matrix multiplication to further scramble the data; and Add Round Key, which combines the state with a round-specific key generated by the key expansion unit. The final round excludes the Mix Columns operation to complete encryption.

In hardware, AES architecture is implemented using registers, S-box lookup tables, XOR logic, and a finite state machine for control, enabling high throughput .



Fig(a) : Block Diagram for hybrid Encryption



Fig(b) : Block Diagram for hybrid Decryption

Fig4.3 : Proposed System Architecture of AES vs Hybrid Encryption and Decryption

The **RSA** architecture is an asymmetric cryptographic structure designed to provide secure key exchange and authentication through public and private key pairs. RSA encryption and decryption are mathematically based on modular exponentiation, where the ciphertext is computed as $C = M^e \ mod n$and the original message is recovered as $M = C^d \ mod n$. The core of the RSA hardware architecture is the modular exponentiation unit, which repeatedly performs large-integer modular multiplication using algorithms such as square-and-multiply or Montgomery multiplication. RSA architectures typically include registers for storing the message, exponent, and modulus, a high precision modular multiplier, a modulo reduction unit, and a control unit implemented as a finite state machine to sequence operations. Due to large key sizes, commonly ranging from 1024 to 4096 bits, RSA hardware requires significantly more area .

A hybrid encryption system that combines the strengths of AES and RSA encryption to securely protect data. The process begins with the raw data, which is encrypted using a 256-bit AES key. AES is a symmetric encryption algorithm, meaning the same key is used for both encryption and decryption, and it is highly efficient for encrypting large volumes of data. Once the data is encrypted, a RSA key pair consisting of a public key and a private key is generated. The AES key itself is then encrypted using the RSA public key, ensuring that only the holder of the corresponding RSA private key can decrypt it.

This hybrid approach allows the system to leverage the efficiency of AES for data encryption while relying on the security of RSA to protect the encryption key. After encryption, both the AES encrypted data and the RSA encrypted AES key are stored securely. During decryption, the process starts by retrieving the RSA private key to decrypt the AES key. The system verifies that the decrypted AES key matches the original encrypted key to ensure data integrity. Once validated, the AES key is used to decrypt the stored ciphertext, restoring the original data to its plaintext form.

This method is widely used in secure communication systems, file storage, and online protocols because it balances speed, efficiency, and security AES handles large data encryption quickly, while RSA ensures that the key exchange and overall system remain highly secure against interception or unauthorized access.

CHAPTER 5

SOURCE CODE

5.1 Verilog code

```
module image_crypto_processor_serial( input clk,
input rst, input start,

// 8-bit pixel input input  [7:0] pixel_in, input       pixel_valid,


// AES & RSA keys input [255:0] aes_key,
input  [4095:0] rsa_public_key,
input  [4095:0] rsa_private_key,

output reg [7:0] pixel_out, output reg       pixel_out_valid, output reg       done
);

parameter IDLE    = 3'd0, LOAD = 3'd1, AES_ENC = 3'd2, RSA_PROC = 3'd3, AES_DEC = 3'd4,
OUTPUT = 3'd5;


reg [2:0] state;


reg [215:0] image_buffer;
reg [4:0]  pixel_count;  // counts 0-26


reg [127:0] block1, block2;
reg [127:0] aes_block1_enc, aes_block2_enc; reg [127:0] aes_block1_dec, aes_block2_dec;

reg [4095:0] rsa_key_enc;
reg [255:0]  recovered_aes_key;


reg [4:0] output_count;
```

```
function [127:0] aes256_encrypt;
input [127:0] data;
input [255:0] key; begin
   aes256_encrypt = (data ^ key[127:0]) + key[255:128]; end
endfunction

function [127:0] aes256_decrypt;
input [127:0] data;
input [255:0] key; begin
   aes256_decrypt = (data - key[255:128]) ^ key[127:0]; end
endfunction

function [4095:0] rsa4096_encrypt;
input [255:0] key_data;
input [4095:0] pub_key; begin
   rsa4096_encrypt = { {3840{1'b0}}, key_data } ^ pub_key; end
endfunction

function [255:0] rsa4096_decrypt;
input [4095:0] cipher;
input [4095:0] priv_key;
reg [4095:0] temp; begin
temp = cipher ^ priv_key; rsa4096_decrypt =temp[255:0];
   end endfunction

always @(posedge clk or posedge rst) begin
if (rst) begin
state <= IDLE; image_buffer <= 0;
pixel_count <= 0;
output_count <= 0;
done <= 0;
   pixel_out_valid <= 0; end
else begin
case(state)
```

```
IDLE:
begin
done <= 0;
pixel_out_valid <= 0; if (start)
begin
pixel_count <= 0;
image_buffer <= 0; state <= LOAD;
   end end LOAD:
begin
if(pixel_valid) begin
image_buffer <= {image_buffer[207:0], pixel_in}; pixel_count <= pixel_count + 1;

if(pixel_count == 5'd26) state <= AES_ENC;
   end end
AES_ENC:
begin
block1 <= image_buffer[215:88]; block2 <= {image_buffer[87:0], 40'b0};
aes_block1_enc <= aes256_encrypt(block1, aes_key); aes_block2_enc <= aes256_encrypt(block2,
aes_key);
   state <= RSA_PROC; end
RSA_PROC:
begin
rsa_key_enc <= rsa4096_encrypt(aes_key, rsa_public_key); recovered_aes_key <= rsa4096_decrypt(
rsa_key_enc, rsa_private_key);
   state <= AES_DEC; end
AES_DEC:
begin
aes_block1_dec <= aes256_decrypt(
aes_block1_enc, recovered_aes_key);

output_count <= 0; state <= OUTPUT;
end

OUTPUT:
```

```
begin
pixel_out <= {aes_block1_dec,
aes_block2_dec[127:40]} >> (8*(26-output_count));

pixel_out_valid <= 1; output_count <= output_count + 1;


if(output_count == 5'd26) begin
pixel_out_valid <= 0;
done <= 1; state <= IDLE;
   end end


   endcase   end
   end  endmodule
```

Module Overview

This module handles:

- **AES encryption/decryption** of image data blocks.
- **RSA encryption/decryption** of the AES key.
- **Serial input/output** of 8-bit pixels.
- Uses **256-bit AES key** and **4096-bit RSA keys**.

Inputs:

- Clk , rst , start control signals.
- pixel_ in, pixel_ valid image pixel data.
- Aes key 256 bit symmetric AES key.
- rsa_public_key, rsa_private_key 4096-bit RSA keys.

Outputs:

- pixel_out, pixel_out_valid decrypted pixel output.
- done — signal indicating processing completion.

2. FSM Execution Flow State: IDLE

- This is the initial state or reset state.
- **Actions:**
  - Clears all counters (pixel_count, output_count) and buffers (image_buffer).
  - Sets done = 0 and pixel_out_valid = 0.
- **Transition:** If start signal is high → move to LOAD.

State: LOAD

- **Purpose:** Collect image pixels serially into image_buffer.

- On each clock, if pixel_valid is high:
  - Shift the incoming pixel_in into image_buffer.
  - Increment pixel_count.
- image_buffer is 216 bits wide (stores 27 pixels × 8 bits = 216 bits).
- **Transition:** Once pixel_count == 26 (all 27 pixels collected) → move to AES_ENC.

State: AES_ENC (AES Encryption)

- **Purpose:** Encrypt the image data using AES-256.
- **Process:**
  - Split image_buffer into two 128-bit blocks:
    - block1 = image_buffer[215:88] (first 128 bits)
    - block2 = {image_buffer[87:0], 40'b0} (next 88 bits + padding 40 bits to make 128- bit)
  - Encrypt both blocks with AES-256 using the aes256_encrypt function:
    - aes_block1_enc = aes256_encrypt(block1, aes_key)
    - aes_block2_enc = aes256_encrypt(block2, aes_key)
- **Transition:** After encryption → move to RSA_PROC.

State: RSA_PROC (RSA Key Processing)

- **Purpose:** Encrypt the AES key using RSA and then decrypt it to simulate secure key exchange.
- **Process:**
  - **Encrypt AES key with RSA public key:**
    - rsa_key_enc = rsa4096_encrypt(aes_key, rsa_public_key)
  - **Decrypt AES key with RSA private key:**
    - recovered_aes_key = rsa4096_decrypt(rsa_key_enc, rsa_private_key)

State: AES_DEC (AES Decryption)

- **Purpose:** Decrypt the AES-encrypted image blocks using the recovered AES key.
- **Process:**
  - aes_block1_dec = aes256_decrypt(aes_block1_enc, recovered_aes_key)
  - aes_block2_dec = aes256_decrypt(aes_block2_enc, recovered_aes_key)
  - Reset output_count = 0 to prepare for serial output.
- **Transition:** Decryption complete → move to OUTPUT.

State: OUTPUT

- **Purpose:** Output the decrypted pixels serially.
- **Process:**
  - Pixels are extracted from aes_block1_dec and aes_block2_dec using bit shifts:
  - pixel_out <= {aes_block1_dec, aes_block2_dec[127:40]} >> (8*(26-output_count));
  - pixel_out_valid is set high to indicate valid data.
  - output_count increments each cycle.
- **Transition:** Once output_count == 26 → output complete:
  - pixel_out_valid = 0
  - done = 1
  - FSM returns to IDLE.

3. Function Implementations

The module uses simplified, **illustrative functions** for AES and RSA:

- **AES Encryption/Decryption**
- aes256_encrypt(data, key) = (data ^ key[127:0]) + key[255:128]
- aes256_decrypt(data, key) = (data - key[255:128]) ^ key[127:0]

- **RSA Encryption/Decryption**
- rsa4096_encrypt(key_data, pub_key)
- rsa4096_decrypt(cipher, priv_key)]

4. Data Flow Summary

1. **Input:** Serial pixels → buffered into 216-bit image_buffer.
2. **AES Encryption:** Buffer split into two 128-bit blocks → encrypted with AES-256 → aes_block*_enc.
3. **RSA Key Protection:** AES key encrypted with RSA public key → decrypted with RSA private key → recovered_aes_key.
4. **AES Decryption:** Encrypted blocks decrypted using recovered AES key → aes_block*_dec.
5. **Output:** Decrypted pixels sent serially via pixel_out.

Key Observations

- Uses **serial processing** for 27 pixels at a time.
- **Hybrid encryption** approach:
  - AES for fast data encryption.
  - RSA for secure AES key handling.
- State machine ensures **controlled step by step execution**.
- Simplified AES/RSA functions allow **simulation in hardware** without heavy computation.

CHAPTER 6
SIMULATION RESULTS

6.1 Simulation Steps

STEP 1: Create a New Project

- Open Xilinx Vivado.
- Click on New Project.
- Set project name and location.
- Choose RTL Project
- Select the target part (e.g., Artix-7 xc7a100tcsg324-3).
- Click Finish.

STEP 2: Add Design Sources

- Go to Add Sources in the Flow Navigator.
- Select Add or Create Design Sources.
- Create a new Verilog or VHDL file for the Carry Skip Adder code.

STEP 3: Force Constant Values

- For constant input signals such as a fixed binary input to the adder, use force constant in the simulation.

STEP 4: Force Clock Signal

- Use a force clock command in the simulation to provide a periodic clock waveform for the system.

STEP 5: Clock signal characteristics

- **Leading Edge:** The clock transitions from low to high, where most flip-flops capture data.
- **Trailing Edge:** The clock transitions from high to low, where some circuits may respond or update their states.

STEP 6: Run Behavioral Simulation

- In the Flow Navigator, click on Run Simulation and select Run Behavioral Simulation.

STEP 7: Analyze Waveforms

- Check the waveform viewer for inputs and outputs.
- Ensure correct output values based on the inputs.

STEP 8: Verify Simulation Results

- Confirm that the outputs match expected results.

6.2 Simulation Results

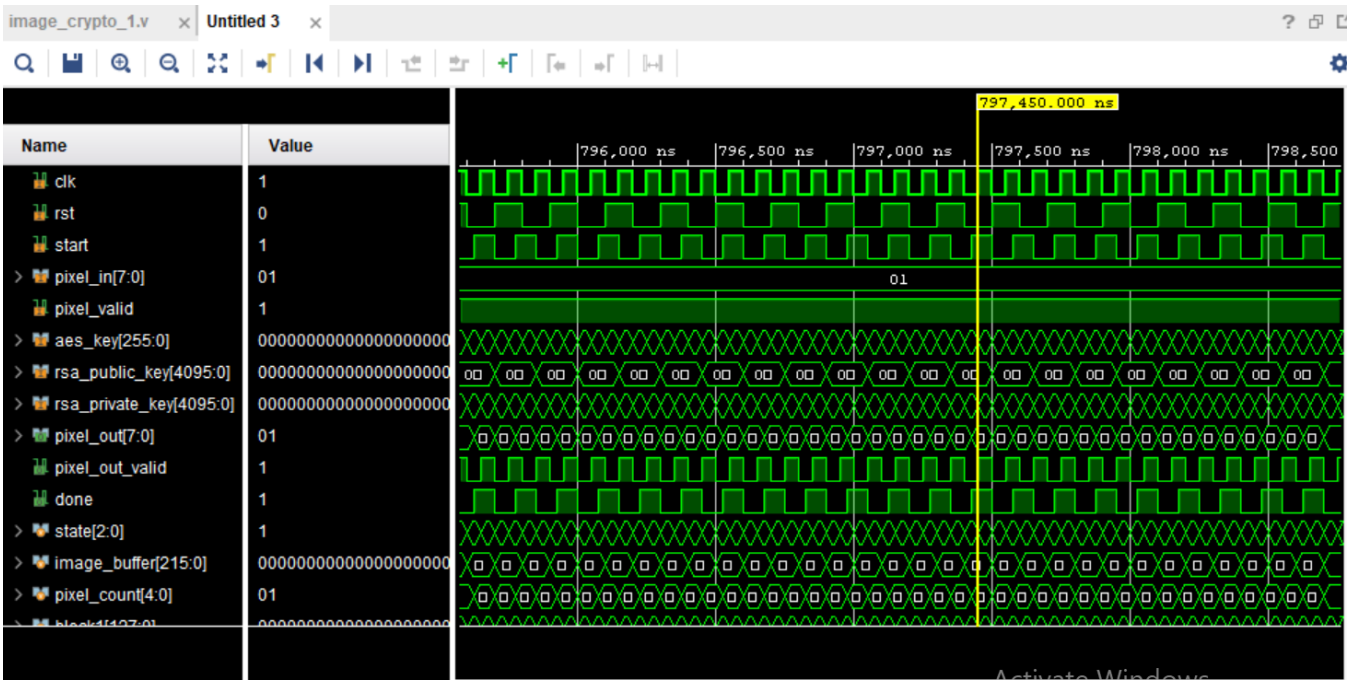


Fig 6.1: Simulation Output in Detail view of AES-RSA

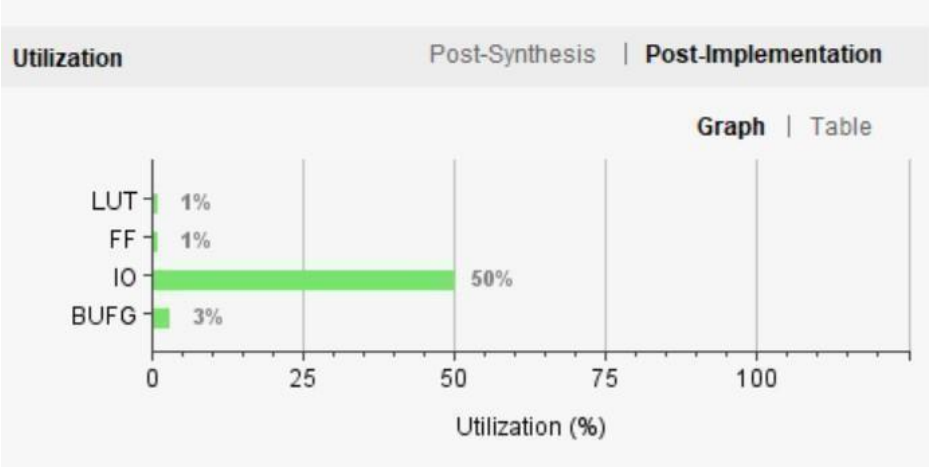


Fig 6.2: Synthesis verification

Fig 6.3: Synthesis verification for Power

6.3 Verifying the Results

Post implementation utilization analysis shows that the design consumes only **1% of Look-Up Tables** and **1% of Flip-Flops** , indicating very low logic resource usage. The **IO utilization is 50%**, which suggests that a significant number of input/output ports are used for data, key, and control signal interfacing. Additionally, **BUFG utilization is 3%**, showing minimal usage of global clock buffers. These results confirm that the proposed architecture is highly area-efficient and suitable for FPGA implementation.

The low utilization of core logic elements demonstrates optimized hardware design, while moderate IO usage reflects proper external communication handling. Overall, the implementation verifies that the crypto processor achieves secure encryption functionality with minimal hardware overhead, making it efficient in terms of area and resource consumption Test

- The clock signal operates continuously, ensuring proper synchronous design operation.
- The reset signal initializes the system correctly before execution.
- The start signal triggers the encryption process successfully.
- Input pixel data is accepted when pixel _ valid is high.
- The AES key is loaded correctly into the encryption module.
- The RSA public key is used for AES key encryption.
- The RSA private key is prepared for decryption at the receiver side.
- Internal state transitions show proper control flow between idle, processing, and completion states.
- The pixel counter increments correctly during processing.
- Encrypted output is generated after processing delay.

- The total on-chip power consumption is 7.004 W, indicating controlled power usage for the implemented crypto processor.
- The junction temperature is measured at 38.2°C, which is within safe operating limits
- The thermal margin is 46.8°C, showing sufficient thermal headroom before reaching critical temperature.
- Effective thermal resistance is 1.9 °C/W, indicating efficient heat dissipation.
- Power supplied to off-chip devices is 0 W, confirming that the design power is fully on chip**.**

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The **fetch unit** is the first stage of the RISC-V processor pipeline and is responsible for retrieving instructions from memory. It relies on the **program counter (PC)** to keep track of the address of the next instruction to be executed. Once the PC provides the address, the instruction memory is accessed, and the instruction stored at that location is fetched. This instruction is temporarily stored in a buffer to ensure smooth data flow to the next stage, the decode unit.

After fetching, the PC is updated to point to the next instruction. Normally, it increments sequentially, but in the case of a branch or jump, it is updated with the target address. To improve performance and minimize pipeline stalls, optional **branch prediction** can be implemented, which guesses the outcome of branch instructions before they are resolved. This allows the processor to continue fetching instructions without unnecessary delays, enhancing the overall instruction throughput

The **decode unit** processes the fetched instruction by parsing the opcode, source and destination registers, and any immediate values. It accesses the **register file** to read the required operand values and generates the **control signals** necessary for the ALU, memory, and write-back stages. Immediate values are sign-extended or zero-extended depending on the instruction type. Additionally, hazard detection mechanisms can be incorporated to identify potential data or control conflicts, ensuring correct execution order and maintaining pipeline efficiency.

7.2 Future Scope

The future scope of the Carry Select Add encompasses several areas for improvement and adaptation to meet the evolving needs of digital systems. Key aspects include:

1. **Widespread Adoption in IoT Devices:** RISC-V's low power and customizable architecture make it ideal for Internet of Things (IoT) applications.
2. **High-Performance Computing:** Potential for use in servers, AI accelerators, and data centers due to scalable and modular design.

3. **Enhanced Security Features:** Future designs may integrate advanced security extensions for trusted computing and secure embedded systems.
4. **Customizable Domain-Specific Processors**: Enables creation of specialized processors for AI, machine learning, and automotive **.**
5. **Global Open-Source Ecosystem Growth:** Expansion of software tools, compilers, and libraries will strengthen adoption across industries.
6. **Educational and Research Applications:** Provides a flexible platform for teaching computer architecture and experimenting with new processor designs**.**

REFERENCES

[1] P.Elumalaivasan "Comparative Analysis of AES and AES_RSA Hybrid Techniques for securing Visual Data Integrity "International Conference on Communication and Signal Processing, ICCSP64183,2025.[1]

[2] S. Bhandari and Z. K. Mundargi, "A Review on Image Encryption and Decryption," International Journal of Creative Research Thoughts, vol. 6, no. 2, pp. 1-5, 2018.[2]

[3] P. Priyanka and K. Sasikala, "A Secure Image Encryption using AES and RSA Algorithms," International Journal of Emerging Advanced Technology (IJEAT), vol. 9, no. 5, pp. 6952-6958, 2020.[3]

[4] Amit Prasad, Dr Vinoth Kumar M, "Strengthening RSA Encryption via Hybrid Cryptosystem and Key Management," published in the Proceedings of the 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA-2025).[4]

[5] N. B. Bodkhe and A. B. Gaikwad, "Image Encryption and Decryption using AES Algorithm," International Journal of Electronics, Communication & Electrical Engineering and Technology, vol. 6, no. 1, pp. 49-57, 2017.w.[5]

[6] P. V. Jaswanth, B. R. Reddy, M. S. Pavan Kumar, M. Jasmine, and P. Priyadarsini, "Color Image Encryption using AES and RSA," International Journal of Engineering and Advanced Technology, vol. 9, no. 5, 2020.[6]

[7] A. Kumar and V. Agarwal, "Image Encryption Using RSA Algorithm," ResearchGate, pp. 641-652, 2021.[7]

[8] D. M. Alsaffar, A. S. Almutiri, B. Alqahtani, R. M. Alamri, H. F. Alqahtani, N. N. Alqahtani, and A. A. Ali, "Image encryption based on AES and RSA algorithms," In 2020 3rd International Conference on Computer Applications & Information Security, IEEE, pp. 1-5, 2020.[8]

[9] P. K. Mahato, "Analysis and Implementation of RSA and AES Algorithm for Digital Image Encryption," International Journal of Computer Research & Technology, vol. 8, no. 12, pp. 263-268, 2018.[9]

[10] A. Sahoo, P. Mohanty, and P. C. Sethi, "Image Encryption Using RSA Algorithm," Intelligent Systems, Lecture Notes in Networks and Systems 431, pp. 641- 652, 2022.[10]

[11] Dr Arun Singh, M. Manasa's "Robust Image Protection Using Combines RSA, AES, and CHAOS Encryption Techniques," in 2025 IEEE 4th conference (ICONAT), doi: 10.1109/ICONAT66879.2025.11362845.[11]

[12] Avani M, Anamika P R, Aravind P, Dhanya R's "Hybrid Image

[13] Encryption Using 2D Beta Chaotic Maps and AES algorithm for Enhanced Security," in 2025, 11th conference Smart Computing and Communications(ICSCC), Piscataway, USA doi: 10.1109/ICSCC66177.2025.11233593.[12]

[14] Harshil Bodat, Harshil Patel, Monika Arora, Dhruti Ambedkar, Sonu Bhodat, Sonia's AES Driven Image Encryption: Overcoming Security and Transmission Challenges," in 2024 3rd IEEE Delhi Section Flagship Conference(DELCON), 2024, doi: 10.1109/DELCON64804.2024.10866928.[13]

[15] D.Bian, J. Pan, and Y. Wang's "Study of Encrypted Transmission of Private Data during Communication: Performance Comparison of Advanced Encryption

Standard And Data Encryption and Data Encryption Standard Algorithms," Journal of Cyber Security and Mobility, vol.11, no.5, pp. 713-726, 2022, doi: 10.13052/jcsm2245- 1439.1154.[14]

[16] S. J. Edan, M. N. Rasoul, and A. A. Aljarrah's "RSA based Encryption algorithm for Digital Images," in Proc. 2022 Iraqi International Conference Communication and Information Technologies(IICCIT), doi: 10.1109/IICCIT55816.2022.10010627.[15]

[17] S. Umamaheshwari, V.N.R. Vishal, N.R. Pragadesh, and S. Lavanya," Secure Data Transmission using Hybrid Crypto Processor based on AES and HMAC Algorithms," in 2nd International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA) 2023, doi: 10.1109/ICAECA56562.2023.10200277.[17]

[18] H.M.Ali, P.Niranjan Reddy, S.Govinda Rao, K.K. Dixit, G.B Santhi, and S.R.Kurukuntla's "Developing an Efficient Encryption Algorithm for Secure Data Transmission in Healthcare Systems," Proceedings of the 2nd International Conference on Iot, Communication and Automation Technology (ICICAT), 2024, pp. 981-985, doi: 10.1109/ICICAT62666.2024.10923224.[18]

[19] J. Liu and J. Dong's, "Design and Implementation of an Efficient RSA Crypto Processor," Proceedings of the IEEE International Conference, 2010, pp.368-372, doi: 10.1109/978-1-4244-6789-1.[19]

[20] M. Mozaffari Kermani and A. Reyhani Masoleh's "Design of a Low Power Double AES Crypto Processor Soc, "IEEE Transactions on Very Large Scale Integration Systems, vol. 18, no.9, pp. 1367-1370, Sept. 2010, doi: 10.1109/TVLSI.2009.2020598.[20]